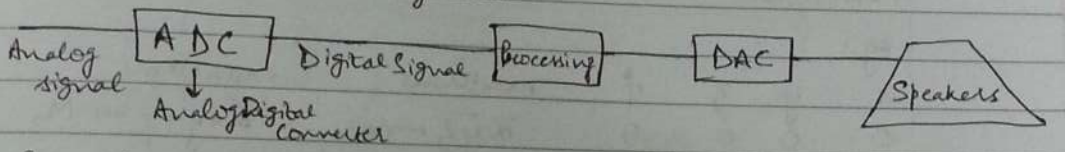
→ Morris Manoe (Digital Design)
→ Fundamentals of Logic Design - Charles H Roth (engage learning)
→ Modern Digital Electronics - R P Jain (for logic families)
→ Bhaskar VHDL

DATE

DIGITAL ELECTRONICS [DE]

- Digital Signals - ON/off - 2 values (High/low)
  Analog " - continuous values
- Advantages of Digital Signals -
  ① Noise immunity
  ② Processing ease - more bits, more values, more capatable of distinguishing b/w 2 values, info to process ↑.
  eg:- in communication systems -



③ Fixed Design
Once a boolean expression is minimised, hardware requirement is min.

- Boolean Algebra.
  Uses 0s and 1s and the operations are slightly different
  + - OR , . - AND

Properties - ① Commutative $x+y = y+x$
② Associative $(x+z)+y = x+(z+y)$
③ Distributive $x(y+z) = xy+xz$
④ Identities $x+0 = x$, $x.1 = x$
⑤ Compliment $x+x' = 1$, $x.x' = 0$
$x.0 = 0$, $x+1 = 1$
⑥ Demorgan's $\overline{(A+B)} = \overline{A}.\overline{B}$ $\overline{A.B} = \overline{A}+\overline{B}$: break the line " change the sign".

⑦ Consensus Theorem (or Implicit factor Theorem)
$xy + \bar{x}z + yz = xy + \bar{x}z$ ["yz is implicit"].
Proof - $xy + \bar{x}z + yz = xy + \bar{x}z + (x+\bar{x})yz$
$= xy + \bar{x}z + xyz + \bar{x}yz$
$= xy(1+z) + \bar{x}z(1+y)$
$= xy + \bar{x}z$.

PAGE

(SumMin) (MaxPro)

DATE

(8) Duality

Interchange 0s by 1s & vice versa and ANDs by ORs & vice versa, and another true expression is obtained.

Dual form of Consensus Theorem →

$(x+y).(\bar{x}+z).(y+z) = (x+y).(\bar{x}+z)$

Proof - either use truth table or expand & rearrange.

- FUNCTIONS.

eg:-

x	y	z	f	minterms	maxterms
0	0	0	0	$\bar{x}\bar{y}\bar{z} \rightarrow m_0$	$x+y+z \rightarrow M_0$
0	0	1	0	$\bar{x}\bar{y}z \rightarrow m_1$	$x+y+\bar{z} \rightarrow M_1$
0	1	0	0	$\bar{x}y\bar{z} \rightarrow m_2$	$x+\bar{y}+z \rightarrow M_2$
0	1	1	1	$\bar{x}yz \rightarrow m_3$	$x+\bar{y}+\bar{z} \rightarrow M_3$
1	0	0	0	$x\bar{y}\bar{z} \rightarrow m_4$	$\bar{x}+y+z \rightarrow M_4$
1	0	1	1	$x\bar{y}z \rightarrow m_5$	$\bar{x}+\bar{y}+\bar{z} \rightarrow M_5$
1	1	0	1	$xy\bar{z} \rightarrow m_6$	$\bar{x}+\bar{y}+z \rightarrow M_6$
1	1	1	1	$xyz \rightarrow m_7$	$\bar{x}+\bar{y}+\bar{z} \rightarrow M_7$

In POS form we take compliment wherever 1 comes since in products, the probability of getting 1 is 1/8 using 3 bits and that of getting 0 is more frequent [ie in A.B.C]

SumMin, MaxPro.

$M_4 \rightarrow$ subscript → obtained as:- eg: $m_3 := 3 \rightarrow 011 = 2^2 \times 0 + 2^1 \times 1 + 2^0 \times 1$

2 raised to the power of its weight

For function 'f' →

① SOP → $f = m_3 + m_5 + M_6 + m_7$

AND-OR realization (and first then 'OR' obtained) ≡ NAND-NAND realization

② POS → $f = M_0 M_1 . M_2 . M_4$

OR-AND realization ≡ NOR-NOR realization

PAGE

DATE

Proof :-

I NAND - NAND ≡ AND - OR

$f = \bar{x}yz + x\bar{y}z + xy\bar{z} + xyz$ = AND-OR realization

Note :- $(x')' = x$

$\Rightarrow (f')' = f$

So $f' = (\bar{x}yz + x\bar{y}z + xy\bar{z} + xyz)$

$= \overline{\bar{x}yz} \cdot \overline{x\bar{y}z} \cdot \overline{xy\bar{z}} \cdot \overline{xyz}$

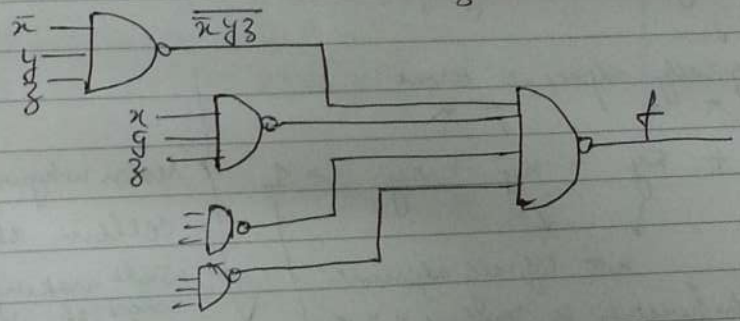
$f = (f')'$

$= \overline{\left(\overline{\bar{x}yz} \cdot \overline{x\bar{y}z} \cdot \overline{xy\bar{z}} \cdot \overline{xyz}\right)}$

Note: And, Not

And, Not → NAND

= NAND - NAND realization.



II OR - AND ≡ NOR - NOR

$f = (x+y+z).(x+y+\bar{z}).(x+\bar{y}+z).(\bar{x}+y+z)$

Again use $(f')' = f$ to prove

PAGE

DATE

Minimization Techniques.

$f = xy + yz + zx + xyz \equiv xy + yz + zx$

xyz — "redundant".

$xy + xyz + yz + zx = xy(1+z) + yz + zx$

min. tech used to ↓ hardware requirement (instead of 4 AND and 4 OR gate we req 3 each).
∴ we use Simplification of boolean expressions.

$xy + x\bar{y} = x(y + \bar{y}) = x \cdot 1 = x$

"logically adjacent terms"

$f = \bar{x}\bar{y} + \bar{x}y + x\bar{y} + xy = 1$

not logically adjacent

the function is "uniformly 1".

reason why we use the cells in that order while making K map concept of logical adjacency is used

1. K-Map (Karnaugh Map). — cells organised in a manner that phy adjacent cells are also logically adjacent.

a) $f(w, x, y, z) =$

$\sum m(0, 1, 2, 4, 5, 8, 9, 10, 13)$

PAGE

DATE

Minimization Techniques.

$f = xy + yz + zx + xyz \equiv xy + yz + zx$

xyz — "redundant".

$xy + xyz + yz + zx = xy(1+z) + yz + zx$

min. tech used to ↓ hardware requirement
(instead of 4 AND and 4 OR gates we req 3 each).
∴ we use Simplification of Boolean Expressions.

$xy + x\bar{y} = x(y + \bar{y}) = x \cdot 1 = x$

"logically adjacent terms"

$f = \bar{x}\bar{y} + \bar{x}y + x\bar{y} + xy = 1$

$\bar{x}\bar{y}$ & $\bar{x}y$ — not logically adjacent.

the function is "uniformly 1".

reason why we use the cells in that order while making K maps. concept of logical adjacency is used.

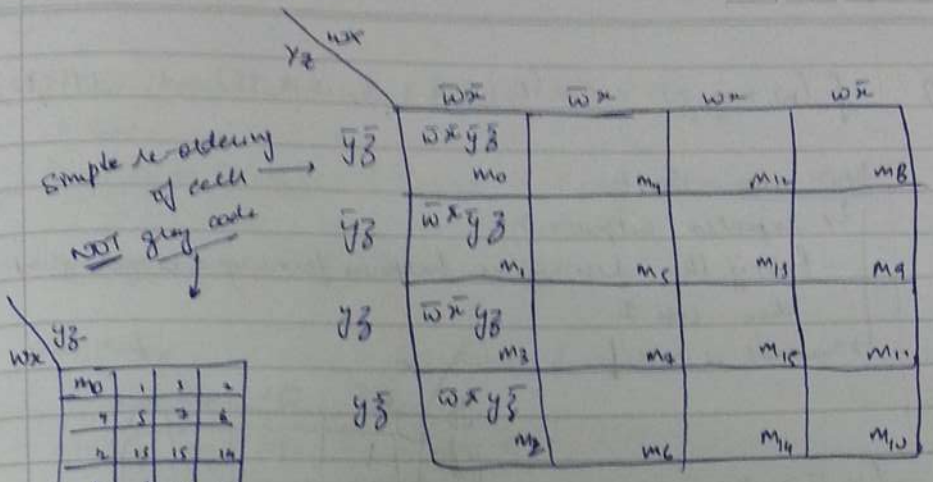
1. K-Map (Karnaugh Map). — cells organised in such a manner that phy. adjacent cells are also logically adjacent.

a) $f(w, x, y, z) =$

$\sum m(0, 1, 2, 4, 5, 8, 9, 10, 13)$

PAGE

DATE



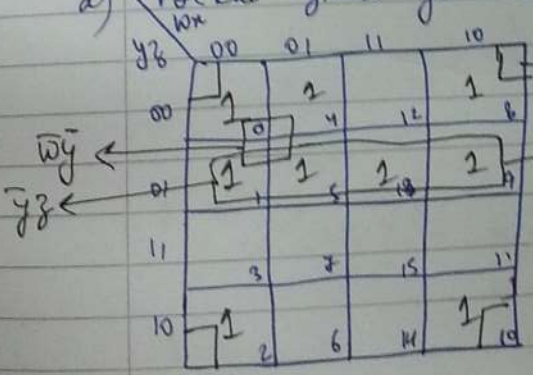
Simple re-ordering of cells → NOT gray code

w	x	y	z	Min terms.
0	0	0	0	$\bar{w}\bar{x}\bar{y}\bar{z} \rightarrow m_0$
0	0	0	1	$\bar{w}\bar{x}\bar{y}z \rightarrow m_1$
0	0	1	0	$\bar{w}\bar{x}y\bar{z} \rightarrow m_2$
0	0	1	1	$\bar{w}\bar{x}yz \rightarrow m_3$
:				
1	1	1	1	$wxyz \rightarrow m_{15}$

By combining logically adjacent cells, cells are taken into consideration (2, 4, 8, 16) so as to eliminate variables.

When all values are 1 so $f = 1$.

a) For the given function,



$f = \bar{x}\bar{z} + \bar{w}\bar{y} + \bar{y}z$

PAGE

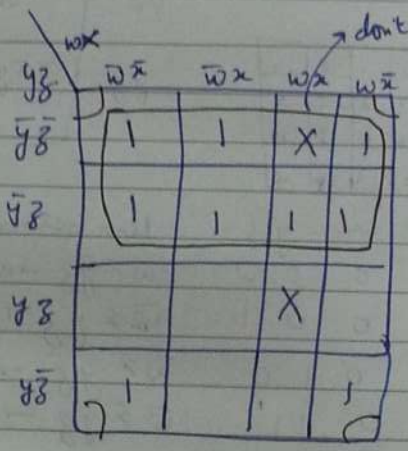
DATE

5) $f(w, x, y, z) = \sum m(0, 1, 2, 4, 5, 8, 10, 9, 13) + d(15, 12)$

Don't care - 15, 12.

↳ expected output = 0
But if the d.c value helps in forming a bigger group then use 1.

↳ "matlab ki dosti"!



$f = \bar{y} + \bar{x}z$

↙ octet

PAGE

DATE

Minimization Techniques (Contd)

1. K Map
2. Tabulation Method (Quine McCluskey Method)
   ↓ Q-M Method.
   used in case of more no. of terms (more than 4).

Suppose there are 2 functions f_1 and f_2 such that :-
- when $f_1 = 1$ then f_2 is also 1
in addition, f_2 is 1 for certain other combinations too
then it is said that f_1 implies f_2.
There is no input combination for which $f_1 = 1$ and $f_2 = 0$.

x	y	z	f_1	f_2
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	1	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

"f_1 is an IMPLICANT of f_2".

since each of the product term implies that the function is 1.

implicants of f_1

$f_1 = \bar{x}yz + xy\bar{z} + xyz = \bar{x}yz + xy$

$f_2 = \bar{x}\bar{y}z + \bar{x}yz + xy\bar{z} + xyz = \bar{x}z + xy$

Subsume

If t_1 and t_2 are product terms such that every literal of t_2 is also a literal of t_1, in that case it is said that t_1 subsumes t_2.

eg:- $t_1 = xy\bar{z}$, $t_2 = xy$ → consists of both $xy\bar{z}$ and xyz since.

This means that t_1 implies t_2.

PAGE

DATE

- Implicant - any product term that implies a function is an implicant of the function. i.e. $f = 1$
- Prime Implicants - It is an implicant of the function from which if any literal is removed, it is no longer an implicant of the function.

eg:- Consider $xy\bar{z}$.

remove x, left with $y\bar{z}$

↳ $f=0$ when $y\bar{z}$ is there but $x=0$.

and " y, " " $x\bar{z}$

↳ $f=0$ " $x\bar{z}$ " " " $y=0$

and " $\bar{z}$, " " xy

↳ $f=1$ " xy " " and $\bar{z}$ is 0 or 1.

- Essential Prime Implicants - minterms having only 1 PI, that PI must be EPI ← considered for sure.

Q* Method. → use K-Maps for 4 variables (in general), here, just to illustrate.

$$f(w,x,y,z) = \sum m(0,1,2,5,8,9,12,14,15)$$

w x y z	Minterms
0 0 0 0	$\bar{w}\bar{x}\bar{y}\bar{z} \rightarrow m_0$
0 0 0 1	$\bar{w}\bar{x}\bar{y}z \rightarrow m_1$
0 0 1 0	$\bar{w}\bar{x}y\bar{z} \rightarrow m_2$
0 0 1 1	$\bar{w}\bar{x}yz \rightarrow m_3$
0 1 0 0	$\bar{w}x\bar{y}\bar{z} \rightarrow m_4$
0 1 0 1	$\bar{w}x\bar{y}z \rightarrow m_5$
0 1 1 0	$\bar{w}xy\bar{z} \rightarrow m_6$
0 1 1 1	$\bar{w}xyz \rightarrow m_7$
1 0 0 0	$w\bar{x}\bar{y}\bar{z} \rightarrow m_8$
1 0 0 1	$w\bar{x}\bar{y}z \rightarrow m_9$
1 0 1 0	$w\bar{x}y\bar{z} \rightarrow m_{10}$
1 0 1 1	$w\bar{x}yz \rightarrow m_{11}$
1 1 0 0	$wx\bar{y}\bar{z} \rightarrow m_{12}$
1 1 0 1	$wx\bar{y}z \rightarrow m_{13}$
1 1 1 0	$wxy\bar{z} \rightarrow m_{14}$
1 1 1 1	$wxyz \rightarrow m_{15}$

PAGE

DATE

→ Divide all the minterms that are representing the function into diff groups. Eg. here:-

0, 1, 2, 5, 8, 9, 12, 14, 15.

index 0. 0 → separate

index 1: 1, 2, 8 } these minterms have only one 1

STAGE 0

index 2: 5, 9, 12 } " " two 1s.

index 3: 14

index 4. 15

→ If diff b/w adj of 2 diff groups is a power of 2 then the 2 values can be combined.

STAGE 1

0, 1 (1)
↓
indicate difference

0, 2 (2)

0, 8 (8)

1, 5 (4)

1, 9 (8)

8, 9 (1)

8, 12 (4)

12, 14 (2)

14, 15 (1)

But (1, 12) cannot be combined coz diff = 11 not a power of 2.

Term with higher index cannot be combined with a term of lower index.

PAGE

DATE

(in adjacent group).

→ Now combine those terms with same difference, and where the diff b/w their corresponding 1st & 2nd terms is a power of 2.

the bracket difference that is common

0, 1, 8, 9 (1, 8)
0, 8, 1, 9 (8, 1)] (A)

[Combined 0,1 (1) and 8,9 (1)
Coz diff = 1 and 8 − 0 = 8 = 2^3, and 9 − 1 = 8 = 2^3.]

bracket diff: 1 − 0 = 1, 9 − 8 = 1
corresponding diff b/w terms 8 − 0 = 8, 9 − 1 = 8

Output —

0, 1 (1) ✓
0, 2 (2) —— B
0, 8 (8) ✓
1, 5 (4) —— C
1, 9 (8) ✓
8, 9 (1) ✓
8, 12 (4) —— D
12, 14 (2) —— E
14, 15 (1) —— F

STAGE 2

These are 6 prime implicants.

→ Now make the prime implicant chart (or table)

A → 0, 1, 8, 9 (1, 8) → cut 1 & 8 → 8 4 2 ✗ / w x y z / 1 → 0 0 0 ✗ / 0 → ø ø ø ø

We can write the binary code of only 4 terms as each, after cancelling 1 & 8 values answer, i.e., here, $\bar{x}\bar{y}$ will yield $\bar{x}\bar{y}$ only.

and take the remaining terms such as we get a 1 in final

B → 0, 2 (2) → take either 0 or 2's code, cancel 2 → 2 → 0 0 ✗ 0 (eliminated 2) → we get $\bar{w}\bar{x}\bar{z}$

C → 1, 5 (4) → $\bar{w}\bar{y}z$ — 1 :- 0 0 0 1
D → 8, 12 (4) → $w\bar{y}\bar{z}$ — 8 :- 1 0 0 0
E → 12, 14 (2) → $wx\bar{z}$ — 12 :- 1 1 0 0
F → 14, 15 (1) → wxy — 14 :- 1 1 1 0

PAGE

While using don't care values, don't mark them in the prime implicant chart. Use those values only in finding the prime implicants.

DATE

Prime Implicants Chart

Minterms → (given in question)

	m_0	m_1	m_2	m_5	m_8	m_9	m_{12}	m_{14}	m_{15}
A → 0,1,8,9 (1,8)	×	×			×	⊗			
B → 0,2 (2)	⊗		⊗						
C → 1,5 (4)		×		⊗					
D → 8,12 (4)					×		×		
E → 12,14 (2)							×	×	
F → 14,15 (1)								×	⊗

Now m_0 has 2 choices, ~~either~~ it can either be covered by A or B.

m_1 " " " " " " " " " " " A or C

m_2 " only 1 choice → B ∴ B must be covered → ESSENTIAL PRIME IMPLICANT

so circle m_2's cross.

m_5 " only 1 choice → C ∴ C too ↑

m_8 — 2 choices - A or D

m_9 → only 1, EPI → A

m_{12} → 2

m_{14} → 2

m_{15} → 1, EPI → F.

Now minimise choices, using EPI.

using A → 0, 1, 8, 9 ✓ ; B → 2 ✓ covered & so on put ticks.

So, using A, B, C, F → all EPIs, just 12 remains.

Now D and E (both of 12) both have 2 crosses

Hence either of it can be used.

∴ $f = A + B + C + F + D$ or $f = A + B + C + F + E$

i.e $f = \bar{x}\bar{y} + \bar{w}\bar{x}\bar{z} + \bar{w}\bar{y}z + wxy$ or $f = \bar{x}\bar{y} + \bar{w}\bar{x}\bar{z} + \bar{w}\bar{y}\bar{z} + wx\bar{z}$

PAGE

DATE

Q//. $f(v,w,x,y,z) = \sum m(4,5,9,11,12,14,15,27,30) + d(1,17,25,$

v,w,x,y,z → 5, 2^5 minterms

v	w	x	y	z	min terms
0	0	0	0	0	m_0
0	0	0	0	1	m_1
0	0	0	1	0	m_2
0	0	0	1	1	m_3
0	0	1	0	0	m_4
0	0	1	0	1	m_5
0	0	1	1	0	m_6
0	0	1	1	1	m_7
0	1	0	0	0	m_8
0	1	0	0	1	m_9
0	1	0	1	0	m_{10}
0	1	0	1	1	m_{11}
0	1	1	0	0	m_{12}
0	1	1	0	1	m_{13}
0	1	1	1	0	m_{14}
0	1	1	1	1	m_{15}
1	0	0	0	0	m_{16}
1	0	0	0	1	m_{17}
1	0	0	1	0	m_{18}
1	0	0	1	1	m_{19}
1	0	1	0	0	m_{20}
1	0	1	0	1	m_{21}
1	0	1	1	0	m_{22}
1	0	1	1	1	m_{23}
1	1	0	0	0	m_{24}
1	1	0	0	1	m_{25}

PAGE

DATE

Q//. $f(v,w,x,y,z) = \sum m(4,5,9,11,12,14,15,27,30) + d(1,17,25,$

5, 2^5 minterms

v	w	x	y	z	min terms
0	0	0	0	0	m_0
0	0	0	0	1	m_1
0	0	0	1	0	m_2
0	0	0	1	1	m_3
0	0	1	0	0	m_4
0	0	1	0	1	m_5
0	0	1	1	0	m_6
0	0	1	1	1	m_7
0	1	0	0	0	m_8
0	1	0	0	1	m_9
0	1	0	1	0	m_{10}
0	1	0	1	1	m_{11}
0	1	1	0	0	m_{12}
0	1	1	0	1	m_{13}
0	1	1	1	0	m_{14}
0	1	1	1	1	m_{15}
1	0	0	0	0	m_{16}
1	0	0	0	1	m_{17}
1	0	0	1	0	m_{18}
1	0	0	1	1	m_{19}
1	0	1	0	0	m_{20}
1	0	1	0	1	m_{21}
1	0	1	1	0	m_{22}
1	0	1	1	1	m_{23}
1	1	0	0	0	m_{24}
1	1	0	0	1	m_{25}

PAGE

v	w	x	y	z	minterms
1	1	0	1	0	m_{26}
1	1	0	1	1	m_{27}
1	1	1	0	0	m_{28}
1	1	1	0	1	m_{29}
1	1	1	1	0	m_{30}
1	1	1	1	1	m_{31}

grouping →
Stage 0

minterm	index
1 ✓	index 1
4 ✓	
5 ✓	index 2
9 ✓	
12 ✓	
17 ✓	
11 ✓	index 3
14 ✓	
25 ✓	
26 ✓	
15 ✓	index 4
27 ✓	
30 ✓	
31 ✓	index 5

1, 4, 8, 9, 11, 12, 14, 18, 17, 25, 26, 27, 30, 31

Stage 1 →

same group:
- 1,5 (4) ← F
- 1,9 (8) ✓
- 1,17 (16) ✓
- 4,5 (1) ← G
- 4,12 (8) ← H

9,11 (2) ✓
9,25 (16) ✓
12,14 (2) ← I
17,25 (8) ✓
11,15 (4) ✓
11,27 (16) ✓
14,15 (1) ✓
14,30 (16) ✓
25,27 (2) ✓
26,27 (1) ✓
26,30 (4) ✓
15,31 (16) ✓
27,31 (4) ✓
30,31 (1) ✓

Stage 2 →

1,9,17,25 (8,16) } A
1,17,9,25 (16,8)
9,11,25,27 (2,16) } B
9,25,11,27 (16,2)
14,30,15,31 (16,1) } D
14,15,30,31 (1,16)
26,27,30,31 (1,4) } E
26,30,27,31 (4,1)
11,15,27,31 (4,16) } C

Till I
4 prime implicants

means that the resultant product term will have 2 variables of weight 8 and 16 eliminated and the common product term can be obtained by writing binary no's and cutting 8 and 16 weights portion.

Can't write (12, 11) as the term with higher index must be higher no. of 1's in binary code.

DATE

Now PI chart, leave don't care values.

PI \ minterms	m_4	m_5	m_9	m_{11}	m_{12}	m_{14}	m_{15}	m_{27}	m_{30}	★ Cost
A (1,9,17,25)			X							4
B (9,11,25,27)			X	X				X		4
C (11,15,27,31)				X			X	X		4
D (14,15,30,31)						X	X		X	4
E (26,27,30,31)								X	X	4
F (1,5)		X								5 (one variable eliminated, 4+1 was ✓ result)
G (4,5)	X	X								5
H (4,12)	X				X					5
I (12,14)					X	X				5

→ NO essential PI. Atleast 2 ways through which each term can be covered. Such a PI chart is "CYCLIC"

Write expression →

$P \rightarrow = \underbrace{(G+H)}_{\text{to cover } m_4 \text{ & } m_{12}} . \underbrace{(F+G)}_{\text{to cover } m_5} . (A+B) . (B+C) . (H+I) . (D+I) . (C+D) . (B+C+E) . (D+E)$

$P = (G + FH) . (B + AC) . (I + DH) . (D + CE)$

$(G+H).(F+G)$

$\hookrightarrow (C+D)(D+E)$

$(B+C+E)$ is made redundant as $(B+C)$ is already there.

$P = BDGI + \ldots\ldots$

covers all minterms 4, 5, 9, 11, 12, 14, 15, 27, 30.

ie all the combination terms we get in 'P'; each is a solution in itself. Thus 'P' includes ALL the possible solutions.

PETRICK'S METHOD.

PAGE

→ In case of Column reduction, dominating column is removed.
→ In case of cyclic PI, use PETRICK'S METHOD.

DATE

Some terms in P require more Prime Implicants, others like BDGI req. less.
L → least, most preferable bcz. min hardware requirement.

- Instead of Petrick's method, another Row Method → ROW REDUCTION METHOD
Row B dominates Row A since B covers all of A & more.
↓ dominating row ↓ dominated row.
↓ can be eliminated.
Now calculate total cost in terms of gates.
like B → eliminates 2 variables, 3 left
total gates req = 3+1 = 4
↙ variables ↘ result to be 'OR'ed for final and.
A = 4 too.
The cost of dominating row should be less than or equal to that of dominated row then the dominated row can be eliminated.
★ Add the 'Cost' column.
Row G and F → F can be eliminated.
Row D & E → E " " "

Now eliminate these 3 rows, make a small chart, make EPI if possible, find result.

	10	8	4	2	1
BDGI	W	X	Y	Z	
B	0	1	0	0	1 · WX YZ

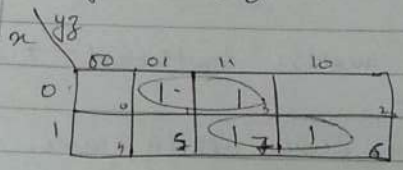
PAGE

DATE

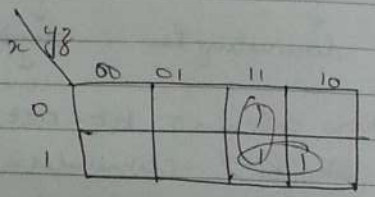
* Multiple Output Functions

$f_1(x,y,z) = \sum m(1,3,6,7)$

$f_2(x,y,z) = \sum m(3,6,7)$



$f_1 = \bar{x}z + xy$

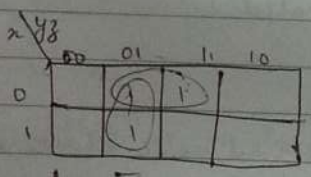


$f_2 = yz + xy$

Sometimes, we don't form biggest groups, instead, we form groups which are common to both the functions so we can use a product term multiple times.

Q// $f_1(x,y,z) = \sum m(1,3,5)$

$f_2(x,y,z) = \sum m(3,6,7)$

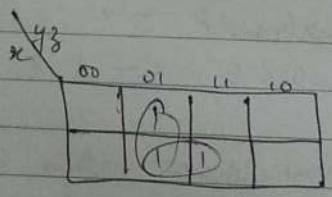


$f_1 = \bar{y}z + \bar{x}z$

or

$f_1 = \bar{x}yz + \bar{y}z$

single pair



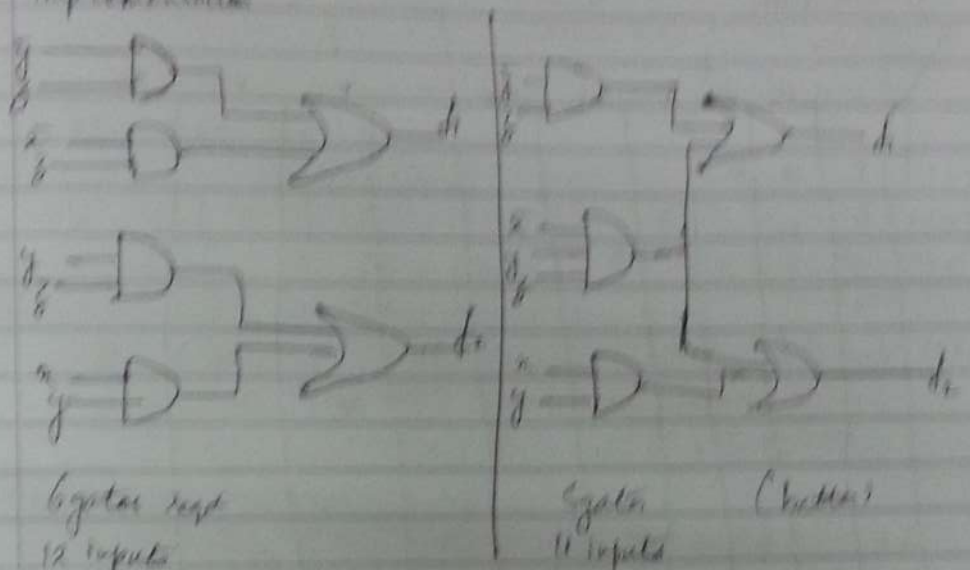
$f_2 = yz + xy$

or

$f_2 = \bar{x}yz + xy$

PAGE

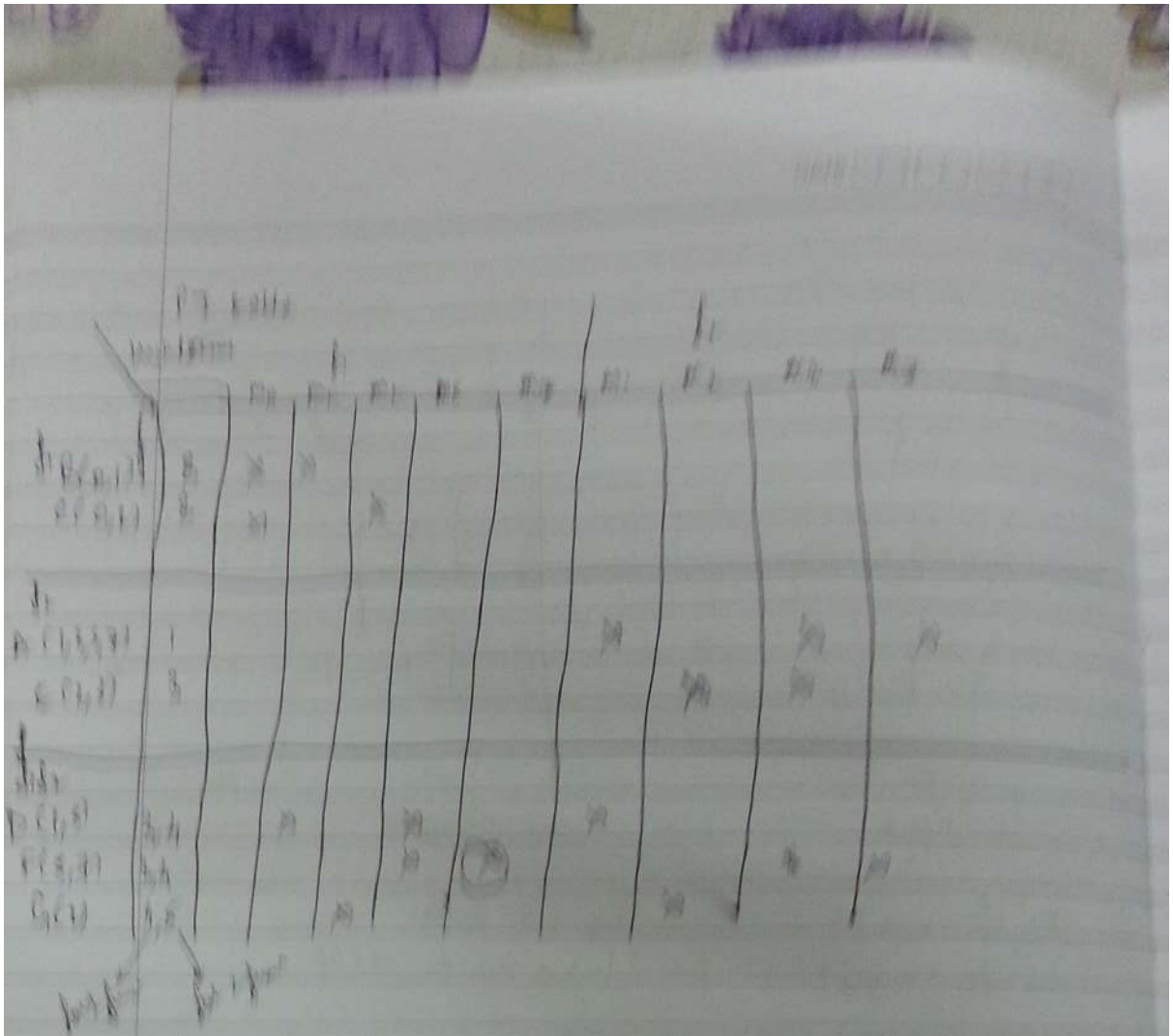
Implementation



* $f_1(x,y,z) = \Sigma m(0,1,2,5,7)$
$f_2(x,y,z) = \Sigma m(1,2,3,7) + d(5)$

Stage 0		Stage 1	
0	f_1 ✓	0,1 (1) f_1 — (A)	In the next stage, retain the common cube removing the f_1 term only that is when two same tag as the resultant tag.
1	$f_1 f_2$ ✓	0,2 (2) f_1 — (B)	
2	$f_1 f_2$	1,3 (2) f_2 ✓	
3	f_2 ✓	1,5 (4) $f_1 f_2$ — (C)	
5	$f_1 f_2$ ✓	2,3, (1) f_2 — (D)	
7	$f_1 f_2$ ✓	3,7 (4) f_2 ✓	
		5,7 (2) $f_1 f_2$ — (E)	

1, 2, 5, 7 (2, 4) — f_2 (A)
1, 5, 3, 7 (4, 2) — f_2



DATE

* Variable entered K-Map (VEM — Variable Entered Mapping technique)

used when certain variables are more frequent. We correlate the output value with the variables.

x	y	z	f	f
0	0	0	f_0	0
0	0	1	f_1	1
0	1	0	f_2	0
0	1	1	f_3	0
1	0	0	f_4	1
1	0	1	f_5	0
1	1	0	f_6	0
1	1	1	f_7	1

function values

$0 - f_0$

$1 - f_1$

$f = f_0\bar{z} + f_1 z$ — function is same as z; when z=0, f=0; z=1, f=1

$f = f_2\bar{z} + f_3 z$ — output has no dependence on z; irrespective of z value, f=0

$f = f_4\bar{z} + f_5 z$

$f = f_6\bar{z} + f_7 z$

xy have same values

$x=0, y=1$

$x=1, y=0$

$x=1, y=1$

So, when we draw the k Maps of such a function such that x & y are the map variables & z is the entered variable then k map size is reduced.

x \ y	0	1
0	$f_0\bar{z} + f_1 z$	$f_2\bar{z} + f_3 z$
1	$f_4\bar{z} + f_5 z$	$f_6\bar{z} + f_7 z$

x \ y	0	1
0	z	0
1	$\bar{z}$	z

w.r.t. y :-

x	z	y	f
0	0	0	f_0
0	1	0	f_1
0	0	1	f_2
0	1	1	f_3
1	0	0	f_4
1	1	0	f_5
1	0	1	f_6
1	1	1	f_7

for f in each when x & z are both.

x \ z	0	1
0	$f_0\bar{y} + f_2 y$	$f_1\bar{y} + f_3 y$
1	$f_4\bar{y} + f_6 y$	$f_5\bar{y} + f_7 y$

Similarly w.r.t. x too it can be done.

PAGE

DATE

* Variable entered K-Map (VEM — Variable Entered Mapping technique)

used when certain variables are more frequent. We correlate the output value with the variables.

x	y	z	f	f
0	0	0	f_0	0
0	0	1	f_1	1
0	1	0	f_2	0
0	1	1	f_3	0
1	0	0	f_4	1
1	0	1	f_5	0
1	1	0	f_6	0
1	1	1	f_7	1

function values

$0 - f_0$

$1 - f_1$

$f = f_0\bar{z} + f_1 z$ — function is same as z; when z=0, f=0; z=1, f=1

$f = f_2\bar{z} + f_3 z$ — output has no dependence on z; irrespective of z value, f=0

$f = f_4\bar{z} + f_5 z$

$f = f_6\bar{z} + f_7 z$

xy have same values

$x=0, y=1$

$x=1, y=0$

$x=1, y=1$

So, when we draw the k Maps of such a function such that x & y are the map variables & z is the entered variable then k map size is reduced.

x \ y	0	1
0	$f_0\bar{z} + f_1 z$	$f_2\bar{z} + f_3 z$
1	$f_4\bar{z} + f_5 z$	$f_6\bar{z} + f_7 z$

x \ y	0	1
0	z	0
1	$\bar{z}$	z

w.r.t. y :-

x	z	y	f
0	0	0	f_0
0	1	0	f_1
0	0	1	f_2
0	1	1	f_3
1	0	0	f_4
1	1	0	f_5
1	0	1	f_6
1	1	1	f_7

for f in each when x & z are both.

x \ z	0	1
0	$f_0\bar{y} + f_2 y$	$f_1\bar{y} + f_3 y$
1	$f_4\bar{y} + f_6 y$	$f_5\bar{y} + f_7 y$

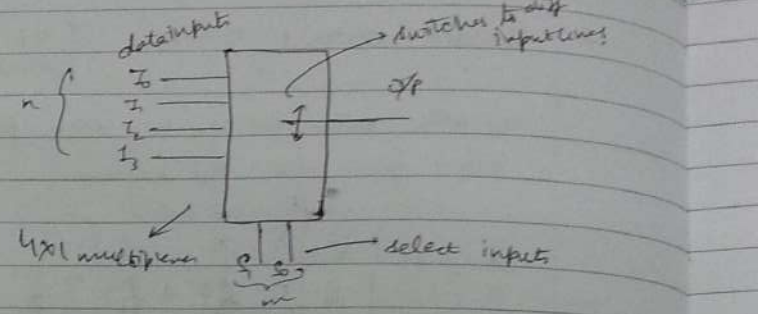
Similarly w.r.t. x too it can be done.

PAGE

DATE

* Multiplexer.

MUX.



$2^m = n$

S_1	S_0	O/P
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Q//

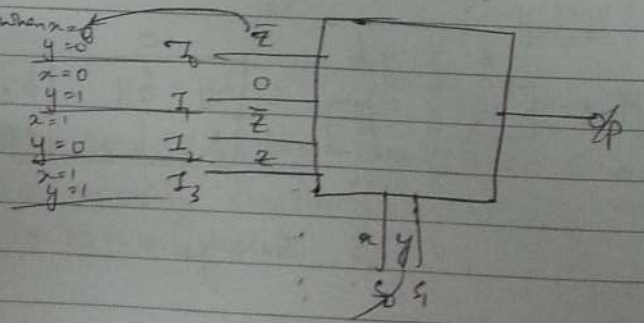
Input variables.		O/P
x	y	f
0	0	0
0	1	1
1	0	1
1	1	0

must apply 0 input to the I_0 line.

Hence we can implement a Boolean expression or a multiplexer.

Suppose we have only a 4x1 multiplexer but we have 3 variables to incorporate, we must use the previous method.

↳ 3 variable func^n can be implemented only with 2 select inputs using EVM.

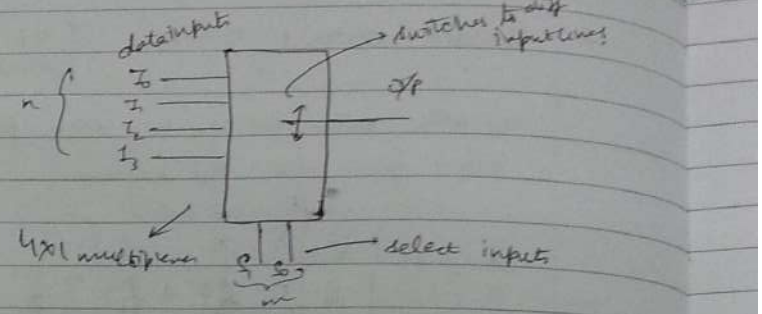


PAGE

DATE

* Multiplexer.

MUX.



$2^m = n$

S_1	S_0	O/P
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Q//

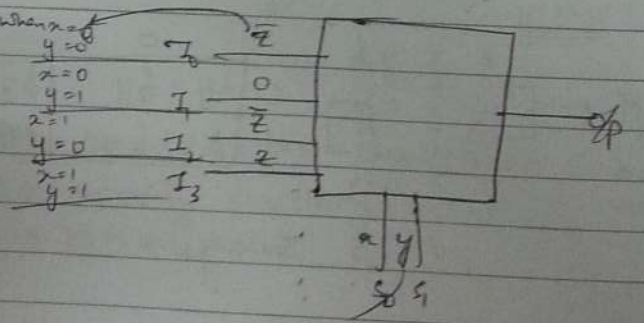
Input variables.		O/P
x	y	f
0	0	0
0	1	1
1	0	1
1	1	0

must apply 0 input to the I_0 line.

Hence we can implement a Boolean expression or a multiplexer.

Suppose we have only a 4x1 multiplexer but we have 3 variables to incorporate, we must use the previous method.

↳ 3 variable func^n can be implemented only with 2 select inputs using EVM.



PAGE

DATE

* Generalization.

f_i	f_j	$f_i\bar{v} + f_j v$	map entry
0	0	$0.\bar{v} + 0v$	$= 0$
0	1	$0.\bar{v} + 1.v$	$= v$
1	0	$1.\bar{v} + 0.v$	$= \bar{v}$
1	1	$1.\bar{v} + 1.v$	$= 1$

Q// Consider a function of 3 variables written as summation of minterms 0, 1, 2, 5. Draw a 2 variable map with the third variable as the map entered variable.

$f(x,y,z) = \Sigma m(0,1,2,5)$

x	y	z	f	f
0	0	0	1	1
0	0	1	1	
0	1	0	1	$\bar{z}$
0	1	1	0	
1	0	0	0	z
1	0	1	1	
1	1	0	0	0
1	1	1	0	

x \ y	0	1
0	1	$\bar{z}$
1	z	0

Q// $f(A,B,x,y,z) = A\bar{x}\bar{y}\bar{z} + \bar{x}\bar{y}z + \bar{x}yz + xyz + Bxy\bar{z}$ +

(5 variables)

xyz appears in each of the given terms, A & B just 2

So use 3 variable map & write A & B as map entered variables.

x \ yz	$\bar{y}\bar{z}$ (0)	$\bar{y}z$ (1)	$y\bar{z}$ (3)	$y\bar{z}$ (2)
$\bar{x}$ 0	A	1	1	0
x 1	0	0	1	B

* Reading the Variable Entered Map.

Treat z and $\bar{z}$ as diff variables and 1 can be written as $(z+\bar{z})$

So 1 can be combined with z, $\bar{z}$

x \ y	0	1
0	$\bar{z}$	
1	z	z+$\bar{z}$

PAGE

DATE

* Generalization.

f_i	f_j	$f_i\bar{v} + f_j v$	map entry
0	0	$0.\bar{v} + 0v$	$= 0$
0	1	$0.\bar{v} + 1.v$	$= v$
1	0	$1.\bar{v} + 0.v$	$= \bar{v}$
1	1	$1.\bar{v} + 1.v$	$= 1$

Q// Consider a function of 3 variables written as summation of minterms 0, 1, 2, 5. Draw a 2 variable map with the third variable as the map entered variable.

$f(x,y,z) = \Sigma m(0,1,2,5)$

x	y	z	f	f
0	0	0	1	1
0	0	1	1	
0	1	0	1	$\bar{z}$
0	1	1	0	
1	0	0	0	z
1	0	1	1	
1	1	0	0	0
1	1	1	0	

x \ y	0	1
0	1	$\bar{z}$
1	z	0

Q// $f(A,B,x,y,z) = A\bar{x}\bar{y}\bar{z} + \bar{x}\bar{y}z + \bar{x}yz + xyz + Bxy\bar{z}$ +

(5 variables)

xyz appears in each of the given terms, A & B just 2

So use 3 variable map & write A & B as map entered variables.

x \ yz	$\bar{y}\bar{z}$ (0)	$\bar{y}z$ (1)	$y\bar{z}$ (3)	$y\bar{z}$ (2)
$\bar{x}$ 0	A	1	1	0
x 1	0	0	1	B

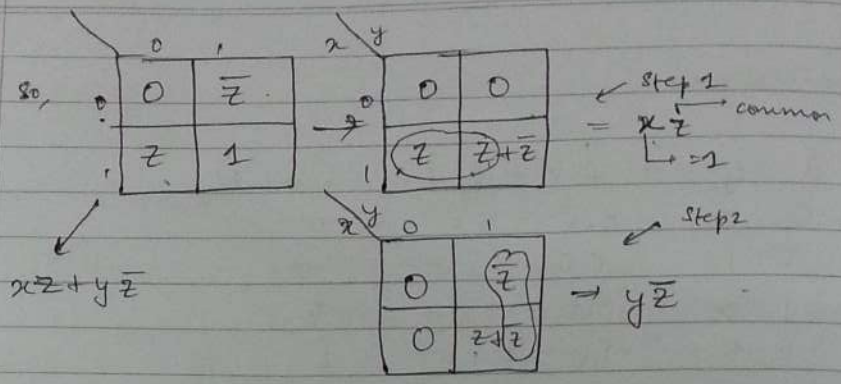
* Reading the Variable Entered Map.

Treat z and $\bar{z}$ as diff variables and 1 can be written as $(z+\bar{z})$

So 1 can be combined with z, $\bar{z}$

x \ y	0	1
0	$\bar{z}$	
1	z	z+$\bar{z}$

PAGE

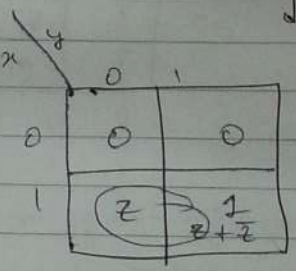


Step 3:- consider remaining 1s (if any) and take rest of the values (vars) as 0.

1 :- treat as Don't care or 1 = z + z̄

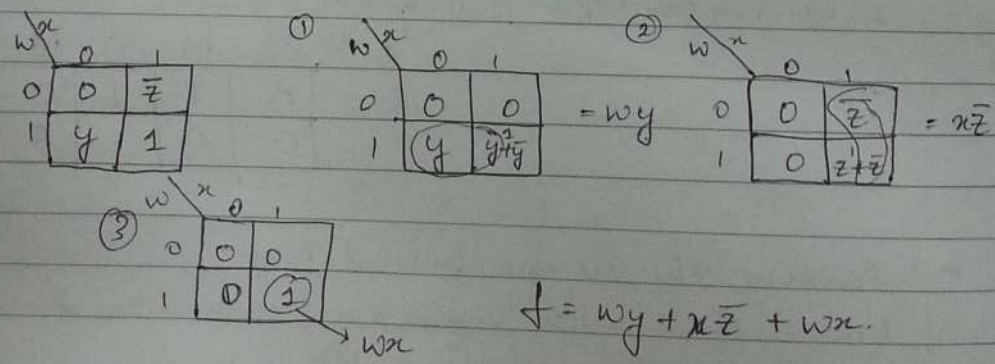
if 1 is fully covered (i.e both z and z̄) have been covered then don't take that 1 again separately.

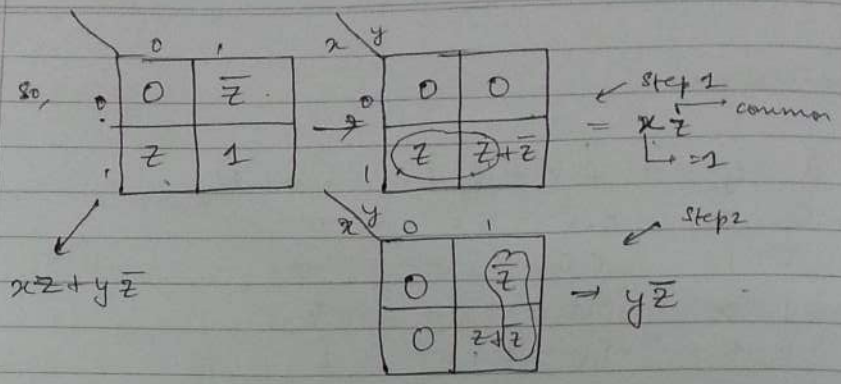
if only z part of 1 is covered, take it (1) again separately.



xz + xy ← for the z̄ part of 1 taken separately.

Ex 1



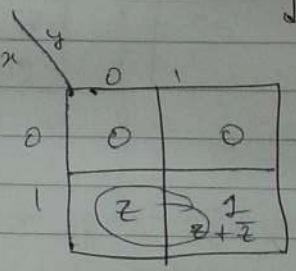


Step 3:- consider remaining 1s (if any) and take rest of the values (vars) as 0.

1 :- treat as Don't care or 1 = z + z̄

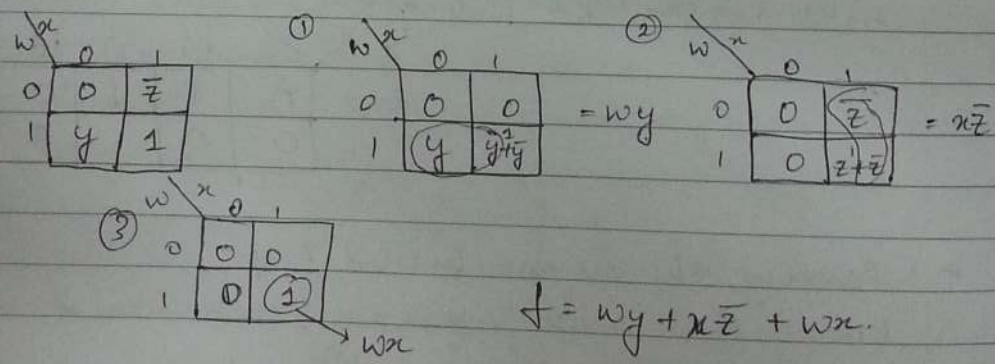
if 1 is fully covered (i.e both z and z̄) have been covered then don't take that 1 again separately.

if only z part of 1 is covered, take it (1) again separately.



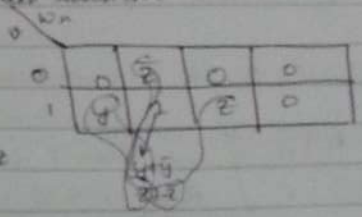
xz + xy ← for the z̄ part of 1 taken separately.

Ex 1



DATE

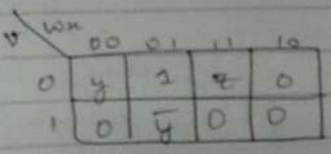
* Had the previous question been such that →

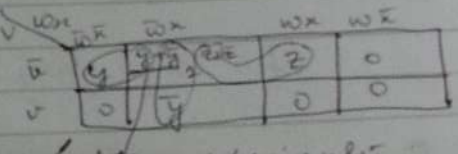


no specific need to denote otherwise to cover 1's in this case.

Now '1' is fully covered using adjacent $\bar{z}$ and z but not using y ($\bar{y}$ remains uncovered) → This doesn't matter, & we would NOT consider '1' separately in this case. once it has been covered - enough.

Q





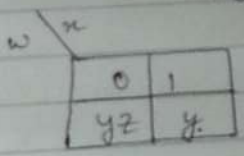
fully covered using y & $\bar{y}$

common part in the 2 cells.

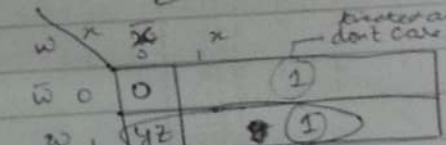
$f = \bar{w}\bar{v}y + \bar{w}\bar{y}x + \bar{v}zx$

for y · for $\bar{y}$ · for z (only for clarity otherwise could've taken the same size)

Q finding product terms.



treating yz

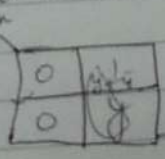


wyz

any term that is part of the product term is taken as 1, — one variable

each of the product term is a separate variable, like yz

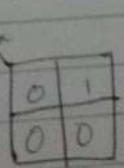
① treating y



$wxyz$ is separate, taken as 0,

yx

② for the $\bar{y}$ uncovered in 1



$\bar{w}x$

$f = wyz + xy + \bar{w}x$

as $y = y($

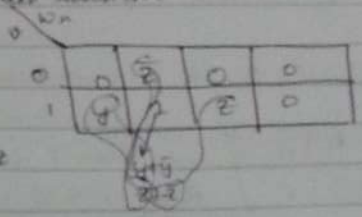
yz

y

Q.

DATE

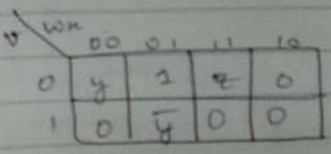
* Had the previous question been such that →

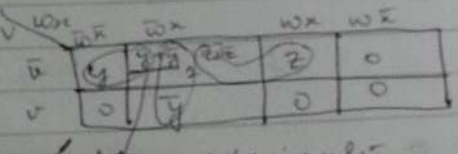


no specific need to denote otherwise to cover 1's in this case.

Now '1' is fully covered using adjacent $\bar{z}$ and z but not using y ($\bar{y}$ remains uncovered) → This doesn't matter, & we would NOT consider '1' separately in this case. once it has been covered - enough.

Q





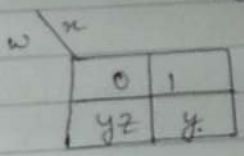
fully covered using y & $\bar{y}$

common part in the 2 cells.

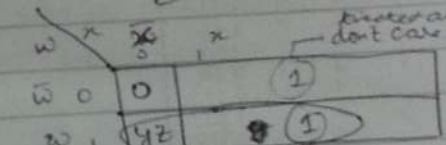
$f = \bar{w}\bar{v}y + \bar{w}\bar{y}x + \bar{v}zx$

for y · for $\bar{y}$ · for z (only for clarity otherwise could've taken the same size)

Q finding product terms.



treating yz

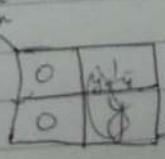


wyz

any term that is part of the product term is taken as 1, — one variable

each of the product term is a separate variable, like yz

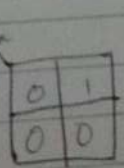
① treating y



$wxyz$ is separate, taken as 0,

yx

② for the $\bar{y}$ uncovered in 1



$\bar{w}x$

$f = wyz + xy + \bar{w}x$

as $y = y($

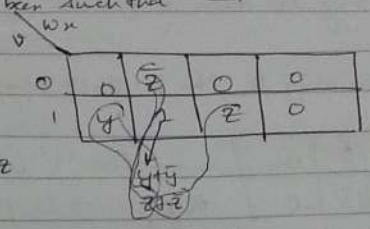
yz

y

Q.

DATE

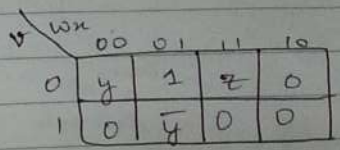
* Had the previous question been such that →

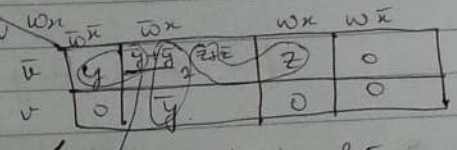


Now '1' is fully covered using adjacent $\bar{z}$ and z but not using y / $\bar{y}$ remains uncovered) → This doesn't matter, & we would NOT consider '1' separately in this case. Once it has been covered - enough.

we specifically need to denote a term only to cover a '1' in this case.

Q.





fully covered using y & $\bar{y}$

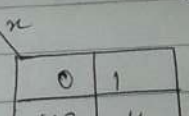
common part in the 2 cells.

$f = \bar{w}\bar{v}y + \bar{w}\bar{y}x + \bar{v}zx$

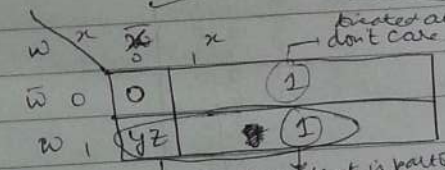
for y · for $\bar{y}$ · for z

(only for clarity between cells of same size)

Q. involving product terms.



① treating yz



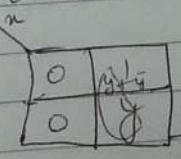
treated as don't care

wyz

any term that is part of the product term is taken as 1. - one variable.

each of the product term is a separate variable, like yz'

② treating y



$wxyz'$ is separate, taken as 0,

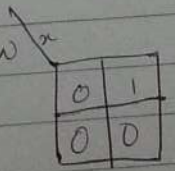
yx

as $y = y(z+\bar{z})$

$= yz + y\bar{z}$

yz this $y = 1$

③ for the y uncovered in 1



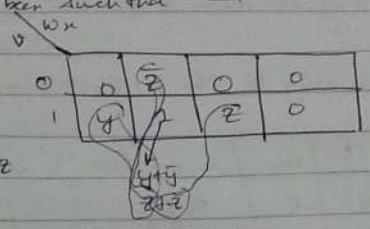
$\bar{w}x$.

$f = wyz + xy + \bar{w}x$.

PAGE

DATE

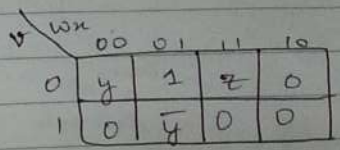
* Had the previous question been such that →

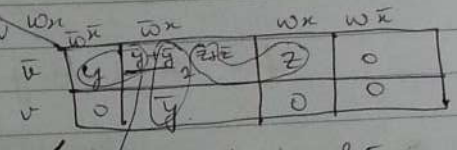


Now '1' is fully covered using adjacent $\bar{z}$ and z but not using y / $\bar{y}$ remains uncovered) → This doesn't matter, & we would NOT consider '1' separately in this case. Once it has been covered - enough.

we specifically need to denote a term only to cover a '1' in this case.

Q.





fully covered using y & $\bar{y}$

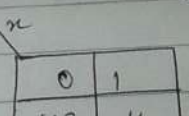
common part in the 2 cells.

$f = \bar{w}\bar{v}y + \bar{w}\bar{y}x + \bar{v}zx$

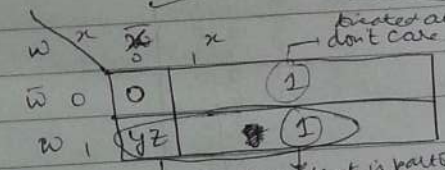
for y · for $\bar{y}$ · for z

(only for clarity between cells of same size)

Q. involving product terms.



① treating yz



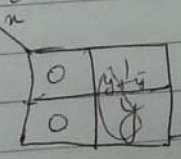
treated as don't care

wyz

any term that is part of the product term is taken as 1. - one variable.

each of the product term is a separate variable, like yz'

② treating y



$wxyz'$ is separate, taken as 0,

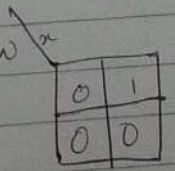
yx

as $y = y(z+\bar{z})$

$= yz + y\bar{z}$

yz this $y = 1$

③ for the y uncovered in 1



$\bar{w}x$.

$f = wyz + xy + \bar{w}x$.

PAGE

DATE

involving the sum & product terms.

Give priority to the product term.

w \ x	0	1
0	1	0
1	yz	0

① the '1' is taken coz $\bar{y}+z$ → this variable is common in yz too

$\bar{x}yz$

w \ x	0	1
0	$\bar{y}$	$\bar{y}$
1	0	0

② $\bar{w}\bar{y}$

$f = \bar{x}yz + \bar{w}\bar{y}$

w \ x	0	1
0	$\bar{y}+z$	$\bar{y}$
1	yz	0

$\bar{y}+z = \bar{y}+z(y+\bar{y})$
$= \bar{y}+zy+\bar{y}z$
$= \bar{y}(z+1)+zy$
$= \bar{y}+zy$

$\bar{y}$ covers in ② — zy already covered in ①

∴ don't take 2 separately

* INCOMPLETELY SPECIFIED FUNCTIONS

(involving don't care terms)

w	x	y	z	$f = \bar{z}f_i + zf_j$
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

PAGE

DATE

involving the sum & product terms.

Give priority to the product term.

w \ x	0	1
0	1	0
1	yz	0

① the '1' is taken coz $\bar{y}+z$ → this variable is common in yz too

$\bar{x}yz$

w \ x	0	1
0	$\bar{y}$	$\bar{y}$
1	0	0

② $\bar{w}\bar{y}$

$f = \bar{x}yz + \bar{w}\bar{y}$

w \ x	0	1
0	$\bar{y}+z$	$\bar{y}$
1	yz	0

$\bar{y}+z = \bar{y}+z(y+\bar{y})$
$= \bar{y}+zy+\bar{y}z$
$= \bar{y}(z+1)+zy$
$= \bar{y}+zy$

$\bar{y}$ covers in ② — zy already covered in ①

∴ don't take 2 separately

* INCOMPLETELY SPECIFIED FUNCTIONS

(involving don't care terms)

w	x	y	z	$f = \bar{z}f_i + zf_j$
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

PAGE

DATE

z	f_i	f_j	Map Entry
0	0		$\bar{z}.0 + z.0 = 0$
0	1		$\bar{z}.0 + z.1 = z$
0	—		$0.\bar{z} + —.z = z, 0$
[don't care			When don't care = 1 → When don't care = 0]
1	0		$1.\bar{z} + 0.z = \bar{z}$
1	1		$1.\bar{z} + 1.z = 1$
1	—		$1.\bar{z} + —.z = 1, \bar{z}$
—	0		$—.\bar{z} + 0.z = 0, \bar{z}$
—	1		$—.\bar{z} + 1.z = 1, z$
—	—		$—.\bar{z} + —z = —$

Reading the map. **RULES**

Step 1. Form an optimal collection of terms for all entries that consist of a single literal using a '—' for don't care, 1s and double entries with constant part 1 as don't care.
In addition, double entries having a zero constant part can be used as don't care for subcubes (combinations) that agree with the literal part of the double entry.
The subcubes must be rectangular and must have the dimension $2^a \times 2^b$ and should be minimized in no. & maximized in size.

Step 2. Form a step 2 map as follows.
i) Replace the single literal entries (v and $\bar{v}$) by 0.
ii) Retain the single 0 and don't care entries.
iii) Replace each single 1 entry by a '—' (don't care) if it was completely covered in step 1. Otherwise, retain the ~~One~~ 1 entry.
iv) Replace the double entries having a 0 constant part ie $v, 0$ and $\bar{v}, 0$ by a 0.

$v, 0$ and $\bar{v}, 0 \longrightarrow 0$ replaced by.

PAGE

DATE

v) Replace each double entry having a 1 count pair by a '–' (or don't care) if the cell was used in step 3 to form atleast one subcube agreeing with the literal part. Otherwise, replace the double entry having a 1 count pair by 1. $\bar{v}, 1$ or $v, 1$.

The resulting step 2 map only has 0, 1 and don't care entries. Hence, an optimal collection of subcubes for the 1 cells should be determined using the cells containing 'x'es (or –es) as don't care cells.

Q) $f(w,x,y,z) = \Sigma m(3,5,6,7,8,9,10) + d(4,11,12,14,15)$

Using EVM

w	x	y	z	f	$f_i\bar{z} + f_jz$	Map Entry
0	0	0	0	0		0
0	0	0	1	0		
0	0	1	0	0		z
0	0	1	1	1		
0	1	0	0	–	$-.\bar{z} + 1.z$	z, 1
0	1	0	1	1		
0	1	1	0	1		1
0	1	1	1	1		
1	0	0	0	1		1
1	0	0	1	1		
1	0	1	0	1	$1.\bar{z} + -.z$	$\bar{z}$, 1
1	0	1	1	–		
1	1	0	0	–	$-.\bar{z} + 0.z$	$\bar{z}$, 0
1	1	0	1	0		
1	1	1	0	–		0, 1, $\bar{z}$, z, –
1	1	1	1	–		

PAGE

DATE

Making the map →

w \ xy	00	01	11	10
0	0	z	1	z,1
1	1	$\bar{z}$,1	–	$\bar{z}$,0

Reading the map →

only one (001) cell with single variable.

3rd stage in Step 1 –

single var, 1,0, – ← 1

double var terms with 1 → don't care.

w \ xy	00	01	11	10
0	0	z	1	z,1
1	1	–	–	$\bar{z}$,0

yz ↵

Step 2.

used only for $\bar{z}$ in 1st step not $\bar{z}$ incomplete

w \ xy	00	01	11	10
0	0	0	1	1
1	1	1	–	0

$x\bar{w}$ ← (row 0)

$w\bar{x}$ ← (row 1)

? should be don't care

$$f = yz + x\bar{w} + w\bar{x}$$

Q// $f(w,x,y,z) = \Sigma m(0,4,5,6,13,14,15) + d(2,7,8,9)$

w	x	y	z	f	fiz + fj$\bar{z}$	Mapping
0	0	0	0	1 → fi	1.$\bar{z}$ + 0. z	$\bar{z}$
0	0	0	1	0 → fj		
0	0	1	0	X		0, $\bar{z}$
0	0	1	1	0		
0	1	0	0	1		1
0	1	0	1	1		
0	1	1	0	1		1, $\bar{z}$
0	1	1	1	X		
1	0	0	0	X		X
1	0	0	1	X		
1	0	1	0	0		0
1	0	1	1	0		
1	1	0	0	0		z
1	1	0	1	1		
1	1	1	0	1		1
1	1	1	1	1		

w \ xy	00	01	11	10
0	$\bar{z}$	0,$\bar{z}$	1,$\bar{z}$	1
1	X	0	1	z

PAGE

DATE

w \ xy	00	01	11	10
0	$\bar{z}$	$\bar{z}, 0$	$\bar{z}, 1$	1
1	X	0	1	z

Step 1 :-

w \ xy	00	01	11	10
0	$\bar{z}$	X	X	1
1	X	0	1	z

$\bar{w}\bar{z}$, xz

Step 2 :-

w \ xy	00	01	11	10
0	0	0	X	X
1	X	0	1	0

xy

$$f = \bar{w}\bar{z} + xy + xz.$$

Q. Write the POS expression for the same function.
(Now use Os as Xs and it should be treated as Os were treated in SOP)
Xs need not be covered, all Os must be covered.

$$f(w, x, y, z) = \Sigma m(3, 5, 6, 7, 8, 9, 10) + d(4, 11, 12, 14, 15).$$

PAGE

* 9's complement - difference from 9.
9's of 4 = 9 - 4 = 5

DATE

Codes

Numbers - weighted codes
- non weighted code.

* Weighted number system

→ 8 4 2 1 BCD code

0 →	0 0 0 0
1 →	0 0 0 1
2 →	0 0 1 0
3 →	0 0 1 1
4 →	0 1 0 0
5 →	0 1 0 1
6 →	0 1 1 0
7 →	0 1 1 1
8 →	1 0 0 0
9 →	1 0 0 1
10	1 0 1 0
11	1 0 1 1
12	1 1 0 0
13	1 1 0 1
14	1 1 1 0
15	1 1 1 1

generated by valid BCD range.

Sum of weights generates the number.
eg:- 5:- 0 1 0 1
8 4 2 1 = 4+1 = 5

6 invalid nos. :- 6 unused states

'8 4 2 1' & '2 4 2 1'
→ weighted codes.

While performing addition →

```
  '0'110
   0111
   1101  → invalid.
 + 0110
 0001 0011
   1    3
```

'6' is added.
↓
always whenever an invalid BCD number is generated or there exists a carry.

⇒ 2 4 2 1 :- "weights" Sum of weights = 2+4+2+1 = 9
here we can get the 9's complement by simply taking compliment of BCD code.

0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	1 0 1 1
6	1 1 0 0
7	1 1 0 1
8	1 1 1 0
9	1 1 1 1

eg:- 9's complement of 0 → 9
and 0000 1111

* 9's complement of 4 → 5
0100 1011
all bits are complement of each other.

→ "SELF COMPLEMENTING CODE".

PAGE

DATE

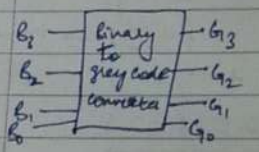
* Non weighted codes - don't depend on bits' weights.

(a) Gray code - only 1 bit is different rest are same.

0 → 00
1 → 01 } same, 1 change
2 → 11 } change, 1 same
3 → 10

0 → 000
1 → 001 } only 1 bit changes
2 → 011 } "
3 → 010 } "
Plane of symmetry
4 → 110 } "
5 → 111 } "
6 → 101 } "
7 → 100 } "

for further values, add a bit.



Q// Know how to change from BCD to gray & viceversa in lab.
Implementing XOR gates

(b) Excess 3. → 0011
add '3' to the normal BCD. ← 8 4 2 1

	E_3	E_2	E_1	E_0
0 →	0	0	1	1
1 →	0	1	0	0
2	0	1	0	1
3	0	1	1	0
4	0	1	1	1
5	1	0	0	0
6	1	0	0	1
7	1	0	1	0
8	1	0	1	1
9	1	1	0	0

9 → 0
0011 → 1100
Again we can obtain the 9's complement by simply complementing the bits. ∴ good for hardware impl.

PAGE

DATE

(c) 2 out of 5 — exactly 2 bits are '1' out of 5 bits. ← error detection capabilities

Certain codes have the capability of detecting an error (only knowing that it has occurred, not locating it).

Certain other codes can correct too, by locating the error bit and taking its complement, hence correcting it.

Error detection & correction. Parity bit is added.

Parity bit : '1' if there are odd no. of ones
0 " " " even " ".

eg:- some given code.

1	0	1	1	1
0	0	1	1	0
1	1	0	1	1
0	1	1	1	1
0	0	1	0	1

→ checking row wise. ↳ called "parity bit".

checking column wise. ↳ called "sum bit" ??

Suppose 0 bit is wrong (the error bit) take its complement then locate (1) and check → row by checking row wise & column by column wise

Had there been only 1 parity ie single parity (either Row or column wise) then the exact location of error bit would not have been easy to find; just the either Row/Column could've been located.

If any bit of the box changes, error can be located.

decimal no.	7 4 2 1 0 → weight except for representation for 0.
	↓ ↓ ↓ ↓ ↓
0	1 1 0 0 0
1	0 0 0 1 1
2	0 0 1 0 1
3	0 0 1 1 0
4	0 1 0 0 1
5	0 1 0 1 0
6	0 1 1 0 0
7	1 0 0 0 1
8	1 0 0 1 0
9	1 0 1 0 0

no of 1s in the code representation is equal to 2.

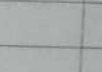
PAGE

DATE

the moment 3 one's are received, the user gets to know that there is an error as there must be only 2 exact no: of ones.

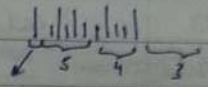
This kind of code is used in ZIP code where a lower bar is used for transmitting 0 and a higher " " " " " 1.

eg



to, code for 5 :- 01010 :- 

used in bar code scanners.

to, 

becomes bar code.

- CHECKSUM. used in error detection (of 1 bit errors).

Code :- 3 9 8 2 4 5 2 3 6 + 8

↑ ↓ ↓ ↓ add a checksum digit.

sum = 42

next closest int divisible by 10, "checksum digit" 50

50 - 42 = 8.

add all digits in code & add a digit in such a manner that it is divisible by 10.

method of checksum used in places where data is stored & error is to be checked without 1 by 1 guesswork.

↓ eg:- if 4 has been coded as 01101 instead of 01001 :- error.

3 ones ✗ 2 ones ✓

so if the error digit is 'e'

we get e + 46 = 50 ← this we had ensured. nearest ÷ by 10

↓ sum of rest

∴ e = 4

eg in EEPROM while writing data to it.

so we get to know that the error digit was 4 and not anything.

PAGE

DATE

Error Detection & Correction

- minimum code distance. M.
(distance in terms of bits. (Different bits b/w any 2 nos.)
no of bits which we have to change one valid code into some other valid code.

eg:- incase of 2 out of 5 → 2 - m
at atleast 2 digits must be changed so that one valid code becomes another valid code.

if D = error detection capability.

$D \leq m-1$

so $D_{2 \text{ out of } 5} = m_{2 \text{ out of } 5} - 1 = 2 - 1 = 1$ → eg using checksum.
so upto one bit errors can be detected.

If m = 3, 2 bit error can be detected

Hamming Code - uses parity bits.

m = no. of information bit

k = no. of parity bits

$m \leq 2^k - k - 1$

eg:- If 4 (m) data bits are there, 3 (k) parity bits must be added only then will the eq" be satisfied. (least value of k that satisfies the eq").

m = 4

$= m_3\ m_2\ m_1\ m_0$

eg:- message 0 1 1 1

7	6	5	4	3	2	1	← Position
m_3	m_2	m_1	P_2	m_0	P_2	P_1	
			$2^2+2^1+2^0$		2^1+2^0	2^0	

$7 = 2^2+2^1+2^0$

2^2+2^1

A Parity must be placed at that pos" which can be written in terms of powers of 2. ie 1, 2, 4, 8...

Now $P_1 \rightarrow 2^0$, select all with 2^0 :- 1, 3, 5, 7

2^1 :- $P_2 \rightarrow$ 2, 3, 6, 7

2^2 $P_3 \rightarrow$ 4, 5, 6, 7

PAGE

DATE

Error Detection & Correction

- minimum code distance, M,
(distance in terms of bits. (Different bits b/w any 2 words)
no. of bits which we have to change one valid code into some other valid code.

eg:- incase of 2 out of 5 → 2 = m
at atleast 2 digits must be changed so that one valid code becomes another valid code.

if D = error detection capability.

$$D \leq m-1$$

so $D_{2 \text{ outof } 5} = m_{2 \text{ outof } 5} - 1 = 2 - 1 = 1$ → eg using checksum.
so upto one bit errors can be detected.
If m = 3, 2 bit error can be detected

Hamming Code - uses parity bits.
m = no. of information bit
k = no. of parity bits

$$m \leq 2^k - k - 1$$

eg:- If 4 (m) data bits are there, 3 (k) parity bits must be added only then will the eqn be satisfied.
(least value of k that satisfies the eqn).

m = 4
$= m_3\ m_2\ m_1 m_0$
eg:- message 0 1 1 1

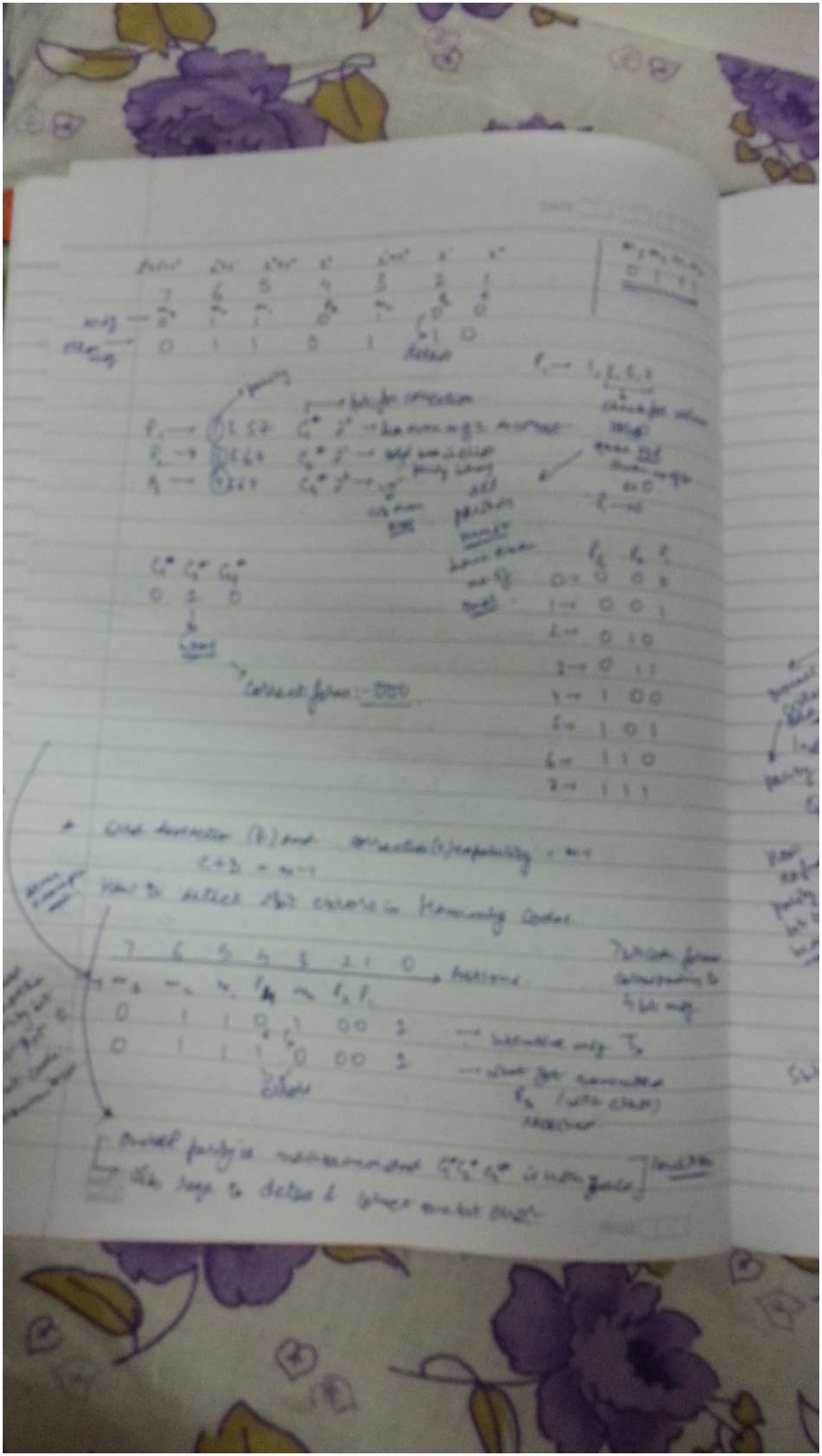
7	6	5	4	3	2	1	Position
m_3	m_2	m_1	P_3	m_0	P_2	P_1	
			2^2+2^0	2^1	2^1+2^0	2^0	

$7 = 2^2+2^1+2^0$ $\quad 2^2+2^1$

A Parity must be placed at that position which can be written in terms of powers of 2. ie 1, 2, 4, 8...

Now $P_1 \rightarrow 2^0$, select all with 2^0 :- 1, 3, 5, 7
2^1 :- $P_2 \rightarrow$ 2, 3, 6, 7
2^2 $P_3 \rightarrow$ 4, 5, 6, 7

PAGE



DATE

	$2^2+2^1+2^0$	2^2+2^1	2^2+2^0	2^2	2^1+2^0	2^1	2^0
	7	6	5	4	3	2	1
msg →	m_3	m_2	m_1	P_3	m_0	P_2	P_1
	0	1	1	0	1	0	0
error msg →	0	1	1	0	1	1	0

detect

$m_3\ m_2\ m_1\ m_0$
0 1 1 1

$P_1 \rightarrow 1, 3, 5, 7$ ↳ check for values

1 ⊕ 1 ⊕ 0 take XOR even no of 1s 0.

$\therefore P_1 \rightarrow 0$

parity → bit for correction.

$P_1 \rightarrow$ (1) 3 5 7 $\quad C_1^* 2^0 \rightarrow$ has even no of 1 so correct

$P_2 \rightarrow$ (2) 3 6 7 $\quad C_2^* 2^1 \rightarrow$ odd no in error

$P_3 \rightarrow$ (4) 5 6 7 $\quad C_4^* 2^2 \rightarrow$ ✓

∴ parity wrong cos even ones.

all parities must have even no. of ones.

	P_3	P_2	P_1
0 →	0	0	0
1 →	0	0	1
2 →	0	1	0
3 →	0	1	1
4 →	1	0	0
5 →	1	0	1
6 →	1	1	0
7 →	1	1	1

$C_4^*\ C_2^*\ C_1^*$
0 1 0
↓
wrong
↓
correct form :- 000.

* Error detection (D) and correction (C) capability = $n-1$

$C+D = n-1$

same example. How to detect 2bit errors in Hamming Codes.

7	6	5	4	3	2	1	0 → Positions.
m_3	m_2	m_1	P_3	m_0	P_2	P_1	
0	1	1	0	1	0	1	→ Intentive msg T_b
0	1	1	1	0	0	1	→ What got transmitted R_b (with error) received.

↳ errors

7 bit code formed corresponding to 4 bit msg.

[overall parity is maintained and $C_4^* C_2^* C_1^*$ is non zero] condition.

↳ also resp to detect & correct one bit error.

PAGE

DATE

$C_4^* C_2^* C_1^* \longrightarrow 111$:- non zero value

[if none of the bits in error it gives 000.]

$P_1 \rightarrow 1357 \rightarrow$ only 1 one $C_1^* :- 1$

$P_2 \rightarrow 2367 \rightarrow$ " 1

$P_4 \rightarrow 4567 \rightarrow$ 3 ones 1

we added another parity bit at the 0th position so that we make the total no. of bits to be even. If already even, no need to add.

incase of detecting & correcting one bit error.

$m_7\ m_6\ m_5\ P_4\ m_3\ P_2\ P_1\ P_0$

0 1 1 0 1 0 0 1 $\longrightarrow$ To

0 1 1 0 1 0 0 0 2 error $\longrightarrow$ P_R

1357

$C_1^* C_2^* C_3^*$ we know that error has occured when overall parity violated i.e the parity that we decided (evens → 0) is NOT coming out to be same in P_R as we're getting

[0 0 0] → each position has even to do all 0

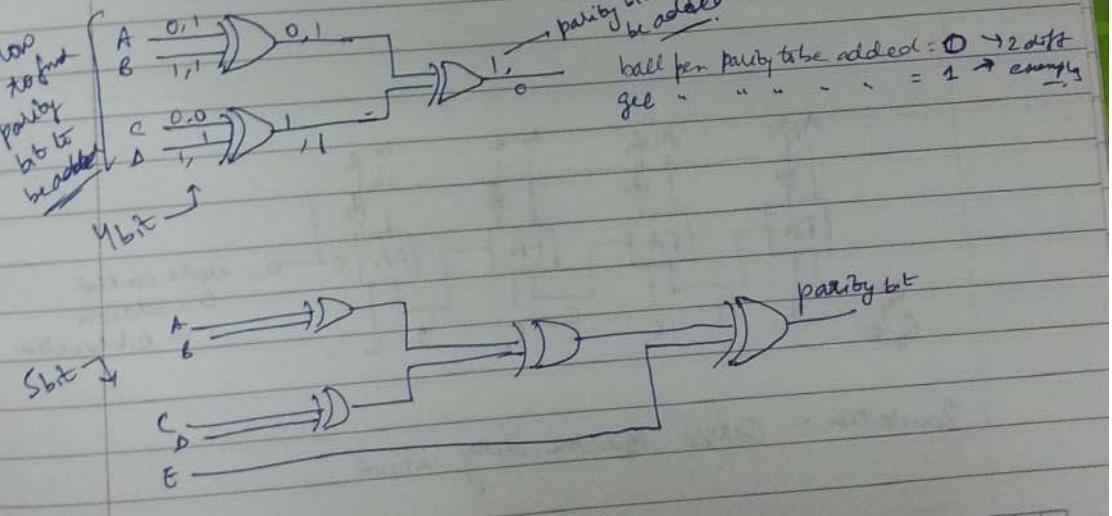
overall parity violation

indicates position → 0th position.

Hence add 0. Address $P_0 P_1 P_2 P_4$ ← 1000

Q// 4 bit info, one bit to be added — parity bit, overall parity - even

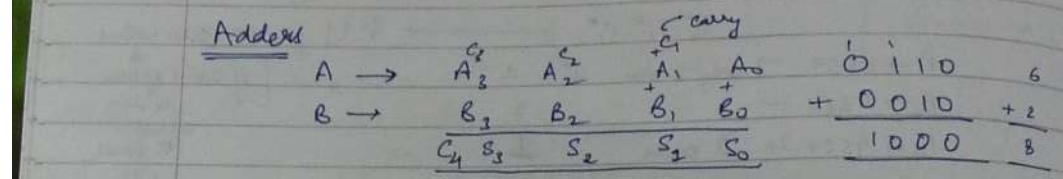
Draw circuit.



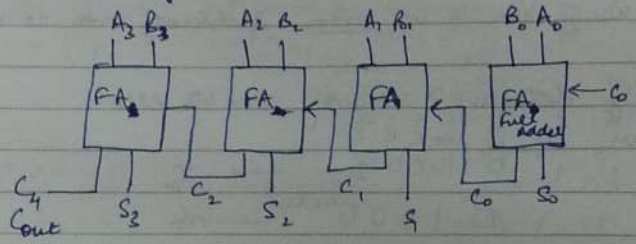
PAGE

DATE

Adders



- Implementation of 4 bit Adder.

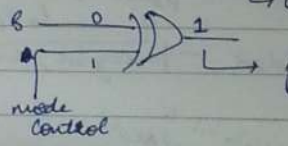


Subtraction

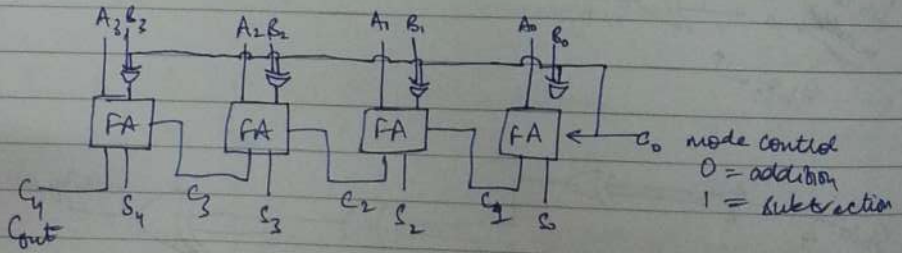
$A - B = A + (-B)$

using complement (1's or 2's)

↳ one's complement +1.



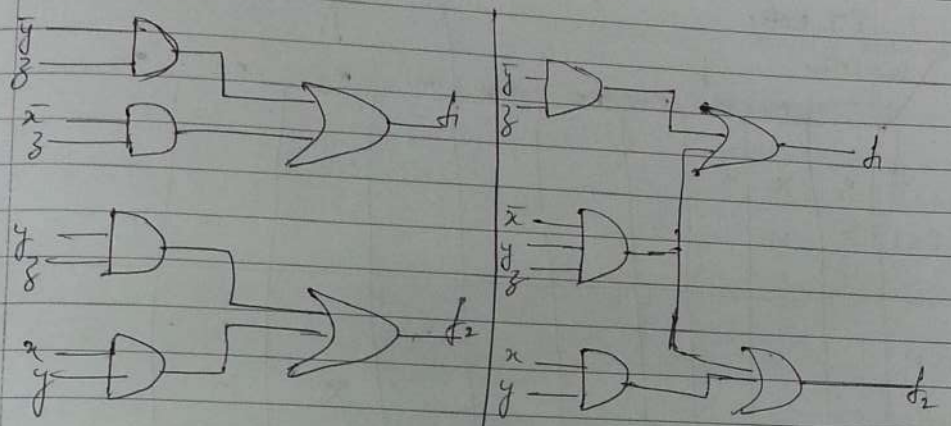
→ if one bit is zero the other input is passed.



Limitation :- Carries Reduction delay increase

DATE

Implementation



6 gates req
12 inputs

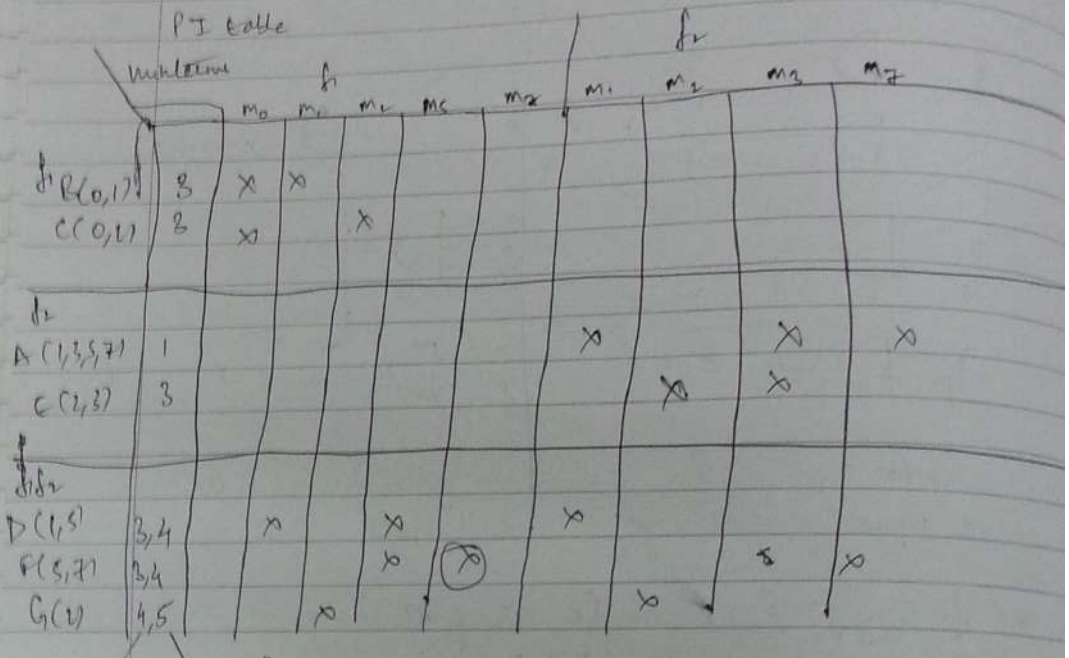
5 gates (better)
11 inputs

* $f_1(x,y,z) = \Sigma m(0,1,2,5,7)$
$f_2(x,y,z) = \Sigma m(1,3,3,7) + d(5)$

Stage 0	Stage 1	
0 f_1 ✓	0,1 (1) f_1 — (B)	In the next stage, retain the common tags remove the other. Tick only that term that has same tag as the resultant tag.
1 $f_1 f_2$ ✓	0,2 (2) f_1 — (C)	
2 $f_1 f_2$	1,3 (2) f_2 ✓	
3 f_2 ✓	1,5 (4) $f_1 f_2$ — (D)	
5 $f_1 f_2$ ✓	2,3, (1) f_2 — (E)	
7 $f_1 f_2$ ✓	3,7 (4) f_2 ✓	
	5,7 (2) $f_1 f_2$ — (F)	

1, 3, 5, 7 (2, 4) — f_2 } (A)
1, 5, 3, 7 (4, 2) — f_2 }

PAGE



→ Tick mark acc. to tag

Use petrick's method for writing all SOPs

$$P = (B_1 + C_1)(B_1 + D_1)(C_1 + G_1)(D + F_1)(F_1)\,\$.$$
$$(A_2 + D_2)(E_2 + G_2)(A_2 + E_2)(A_2 + F_2)$$

Expand & write.

$f_1 =$

all terms having subscript 1.

$f_2 =$

all PIs with subscript 2

After including x_2, the cost of this is modified for f_2 as 1 as this has to be necessarily generated.

DATE

	$2^3+2^2+2^0$	2^3+2^1	2^3+2^0	2^2	2^1+2^0	2^1	2^0
	7	6	5	4	3	2	1
	m_3	m_2	m_1	P_3	m_0	P_2	P_1
msg →	0	1	1	0	1	0	0
error msg →	0	1	1	0	1	1	0

detect

$m_3 m_2 m_1 m_0$
0 1 1 1

$P_1 \rightarrow 1, 3, 5, 7$ — check for values 1 ⊕ 1 ⊕ 0 take XOR even no of 1s 0 0. ∴ $P_1 \rightarrow 0$

parity → bit for correction.

$P_1 \rightarrow$ (1) 3 5 7 C_1^* 2^0 → has even no of 1s so correct

$P_2 \rightarrow$ (2) 3 6 7 C_2^* 2^1 → odd ones in error

$P_3 \rightarrow$ (4) 5 6 7 C_4^* 2^2 → ∴ parity wrong as even ones. all parities must have even no. of ones.

	P_3	P_2	P_1
0 →	0	0	0
1 →	0	0	1
2 →	0	1	0
3 →	0	1	1
4 →	1	0	0
5 →	1	0	1
6 →	1	1	0
7 →	1	1	1

C_1^* C_2^* C_4^*
0 1 0
↓
wrong.
→ Correct form :- 000

* Error deccection (D) and correction (C) capability : $m-1$

$C+D = m-1$

same example. How to detect 2bit errors in Hamming Codes.

7	6	5	4	3	2	1	0 → Positions.
m_3	m_2	m_1	P_3	m_0	P_2	P_1	
0	1	1	0	1	0	0	→ intentive msg T_x
0	1	1	1	0	0	1	→ what got transmitted R_x (with error) received.
			↓ errors				

7 bit code for corresponding to 4 bit msg

add another parity bit as for 0. bit code is transmitted.

overall parity is maintained and $C_1^* C_2^* C_4^*$ is non zero.] condition.

→ also reqs to detect & correct one bit error.

PAGE